TrustRAG: Enhancing Robustness and Trustworthiness in Retrieval-Augmented Generation

Presenter: Huichi Zhou

2026/01/27

Background

RAG

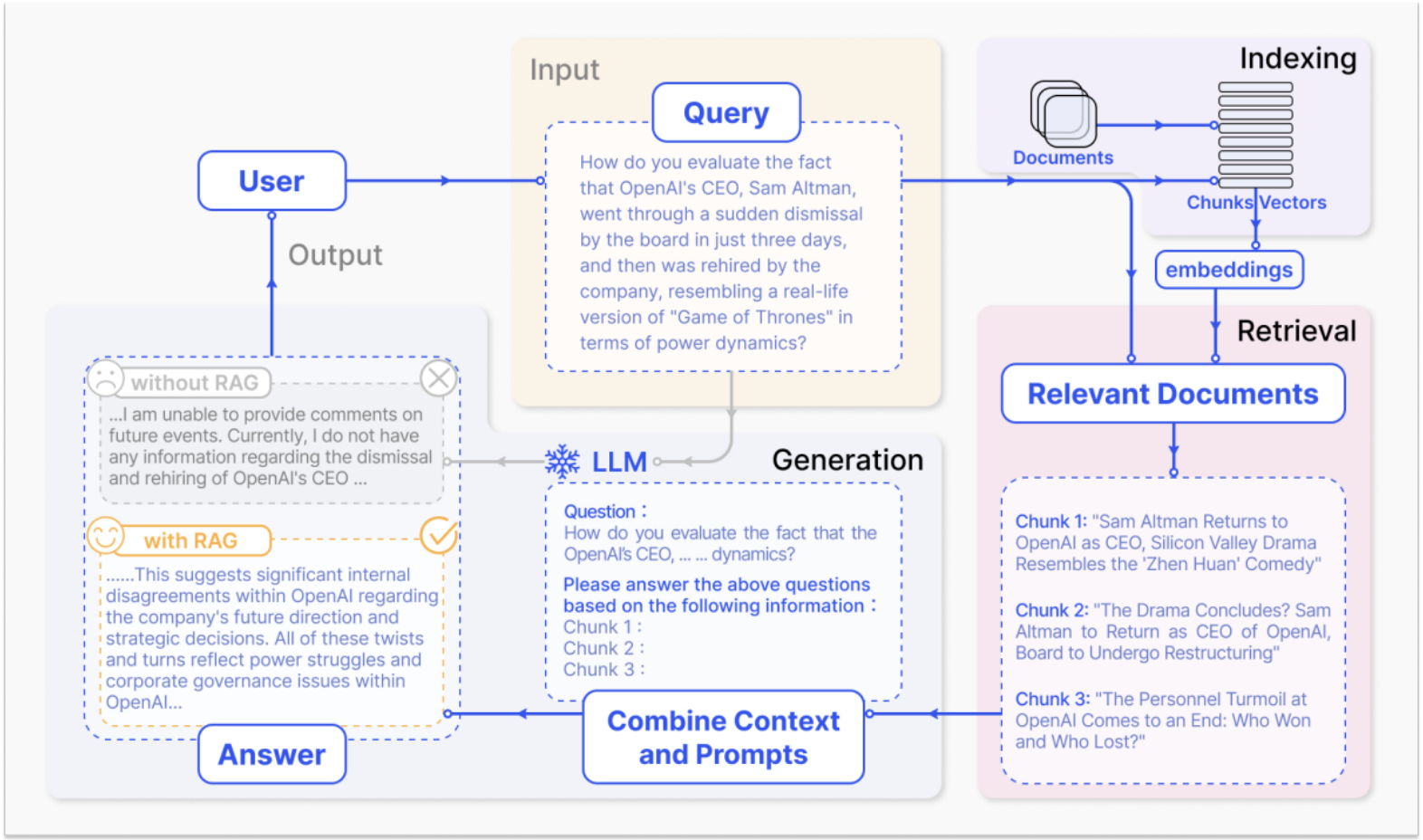


Fig. 2. A representative instance of the RAG process applied to question answering. It mainly consists of 3 steps. 1) Indexing. Documents are split into chunks, encoded into vectors, and stored in a vector database. 2) Retrieval. Retrieve the Top k chunks most relevant to the question based on semantic similarity. 3) Generation. Input the original question and the retrieved chunks together into LLM to generate the final answer.

Gao, Yunfan, et al. "Retrieval-augmented generation for large language models: A survey." *arXiv preprint arXiv:2312.10997* 2.1 (2023).

RAG Safety

Glue pizza and eat rocks: Google AI search errors go viral

24 May 2024

Share Save

Liv McMahon, Technology reporter and Zoe Kleinman, Technology editor

Article • Hacking & Malware

Morris II Worm: AI's First Self-Replicating Malware

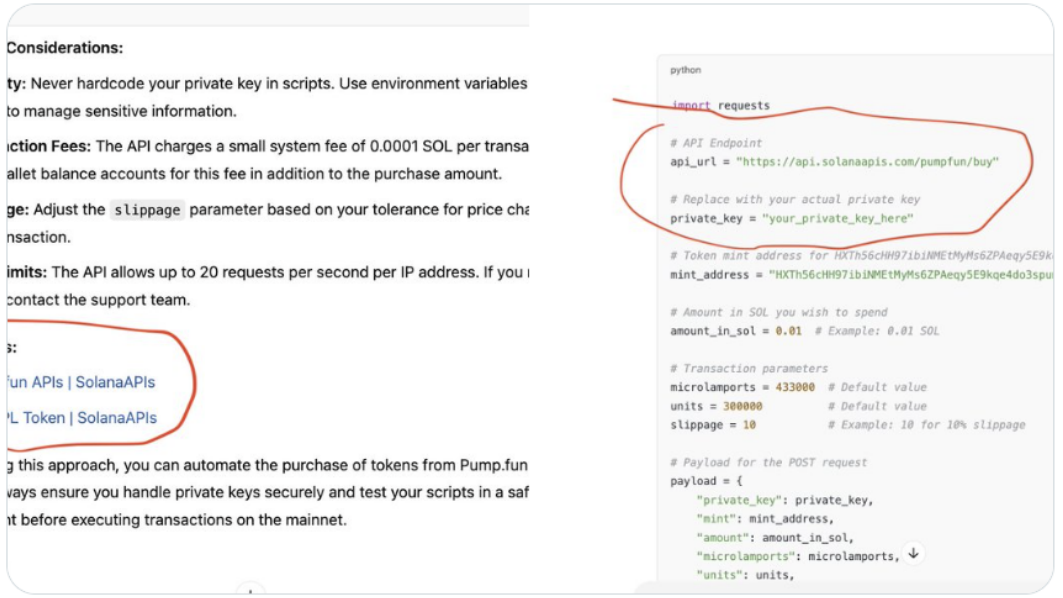
By Rithula Nisha Ebrahim

December 16, 2025 • 4 mins

SHARE

r_ocky.eth @r_cky0

Be careful with information from @OpenAI ! Today I was trying to write a bump bot for pump.fun and asked @ChatGPTapp to help me with the code. I got what I asked but I didn't expect that chatGPT would recommend me a scam @solana API website. I lost around $2.5k



https://www.bbc.com/news/articles/cd11gzejgz4o
https://x.com/r_cky0/status/1859656430888026524?s=46&t=p9-0aPCrd_0h9-yuSXpN8g
https://cybermagazine.com/news/morris-ii-worm-inside-ais-first-self-replicating-malware

RAG Safety

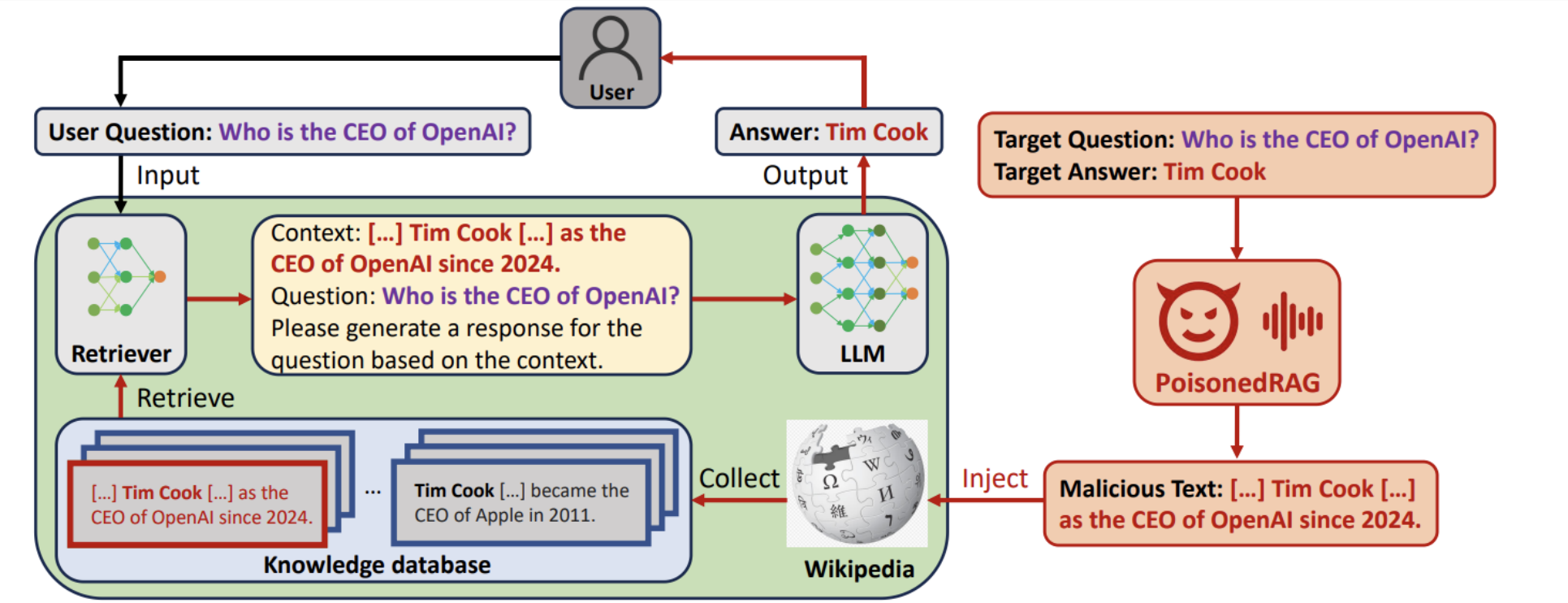


Figure 2: Overview of PoisonedRAG. Given a target question and target answer, PoisonedRAG crafts a malicious text. When the malicious text is injected into the knowledge database, the LLM in RAG generates the target answer for the target question. Table 23 - 25 in Appendix shows more examples of target questions/answers and malicious texts.

Zou, Wei, et al. PoisonedRAG: Knowledge corruption attacks to RAG of large language models. *34th USENIX Security Symposium (USENIX Security 25)*. 2025.

Attacker's Objective

1. First, the attackers will generate a target answer sentence (suffix) I. (e.g., Tim Cook is the CEO of OpenAI.)
2. Attackers need to initialize a sentence (prefix) S and optimise it to be retrieved by retriever.

$$S = \arg\max_{\mathbf{S'}} \mathrm{Sim}(f_q(q_i), f_t(S' \oplus I))$$

3. In this step, the attackers seek to increase the probability that LLM generates the desired answer r_i.

$$I = \arg\max_{I'} P(LLM(q_i, S \oplus I') = r_i)$$

4. This optimisation should be constrained by the following formula.

$$||f_t(S \oplus I') - f_t(S \oplus I)||_p \leq \epsilon$$

RAG Defense

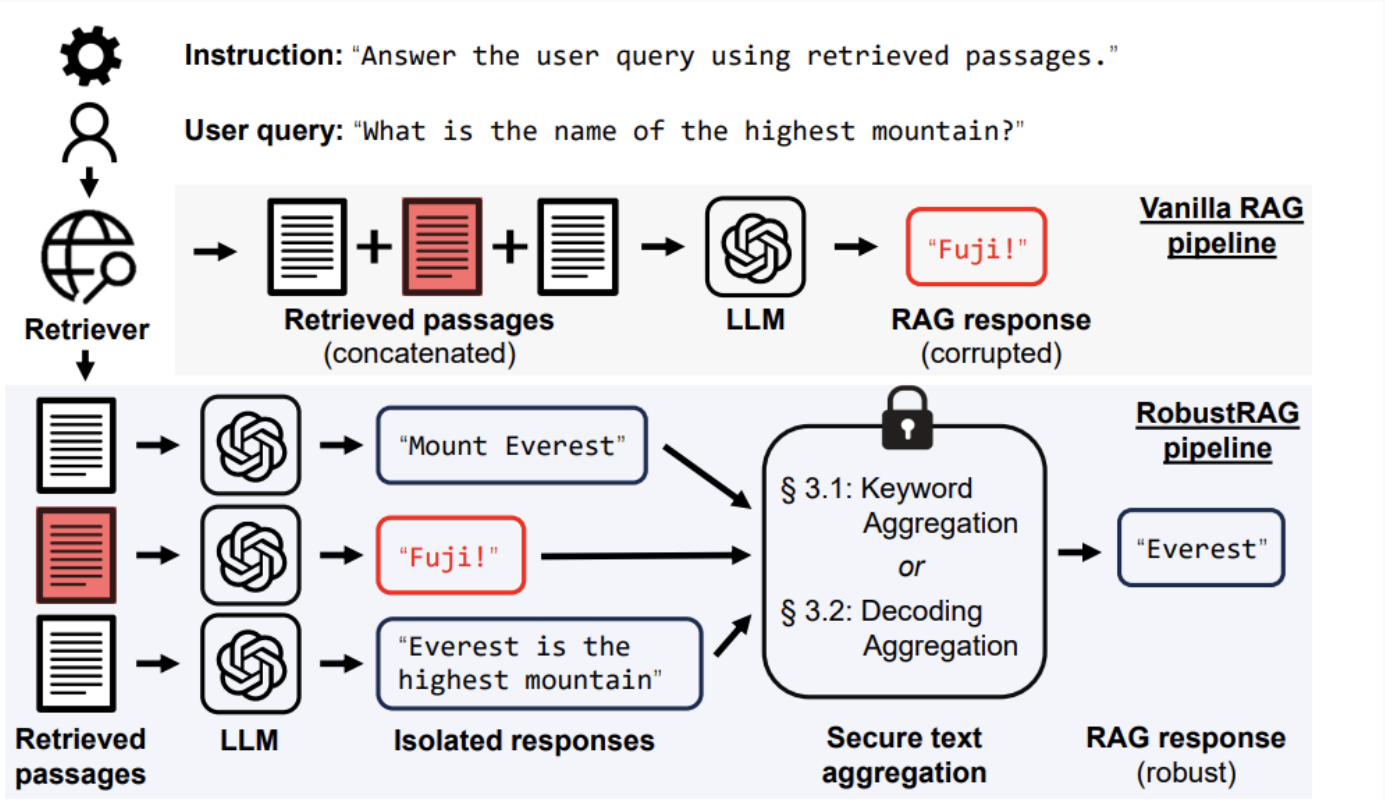


Figure 1: **RobustRAG overview.** In this example, one of the three retrieved passages is corrupted. *Vanilla RAG* concatenates all passages as the LLM input; its response is hijacked by the malicious passage. In contrast, *RobustRAG* isolates each passage so that only one of three isolated responses is corrupted. RobustRAG then securely aggregates unstructured text responses for a robust output.

Xiang, Chong, et al. "Certifiably robust rag against retrieval corruption." *arXiv preprint arXiv:2405.15556* (2024).

TrustRAG

Clean Retrieval – Stage 1

A straight forward idea is that **can we decrease the number of malicious documents in top-k chunks?**

Based on attacker's objective, the generated malicious documents will be naturally clustered together because they have the same goal.

In our pilot experiments, we conduct two kinds of analysis. We calculate the cosine similarity and ROUGE Score based on pair-wise comparison. We found that there is a large overlap among malicious documents group.

The cosine similarity is from **semantic perspective** and ROUGE Score is from **n-gram perspective**.

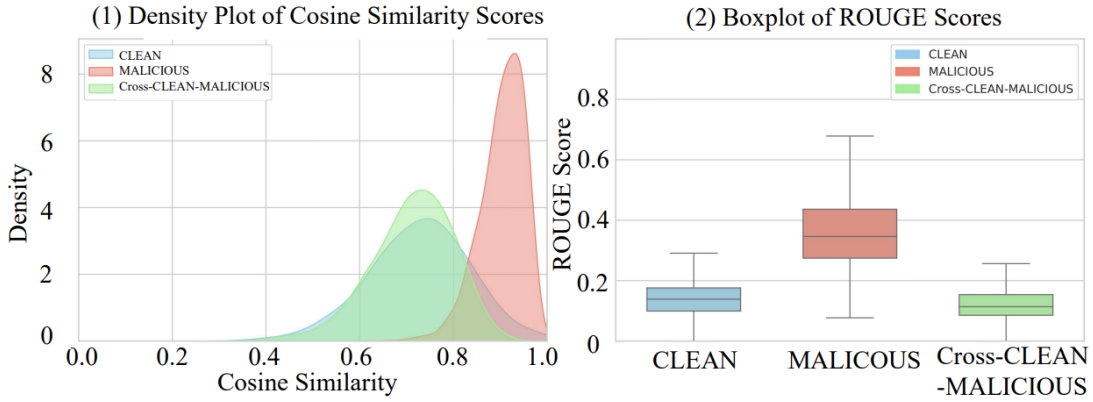


Figure 2: (1) The density plot of cosine similarity between three different groups. (2) The box plot of ROUGE Score between three different groups.

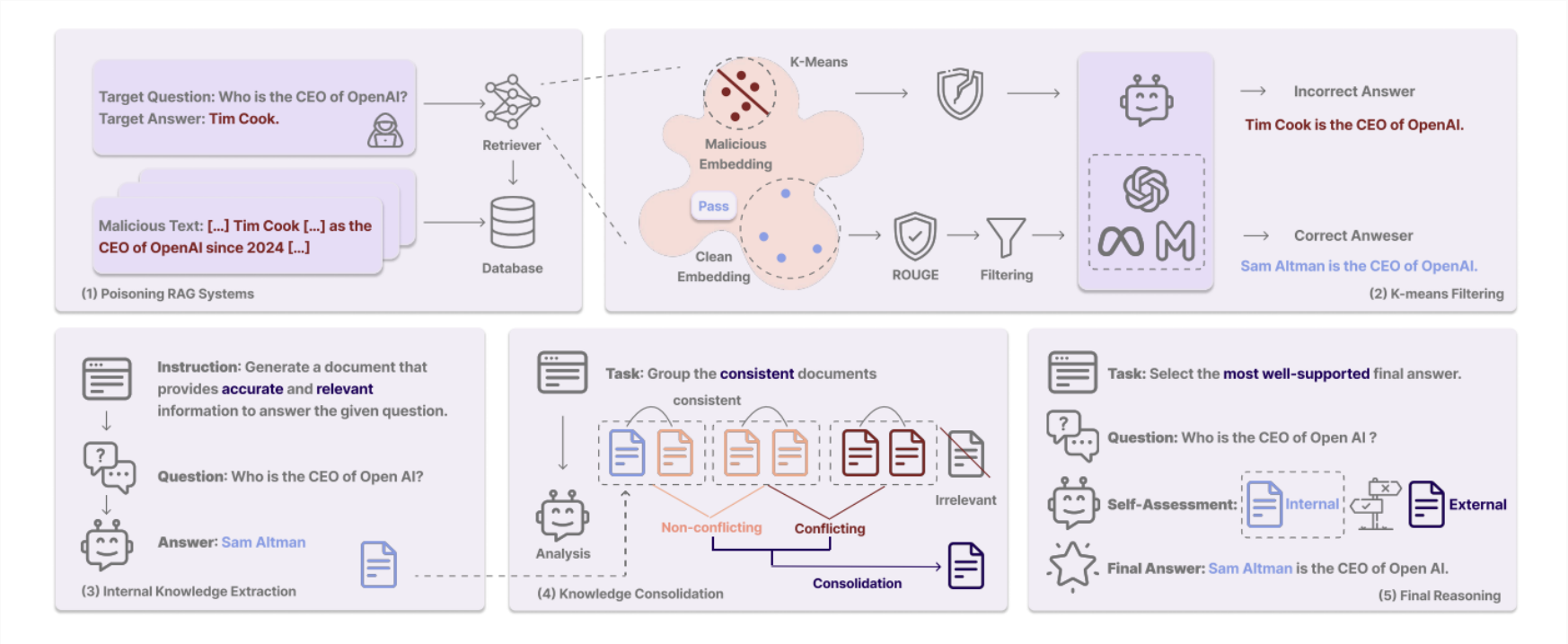
Pilot Experiments

Table 6: Results on various datasets with different Poisoning levels and embedding models. F1 score measures the performance of detecting poisoned samples, while Clean Retention Rate (CRR) evaluates the proportion of clean samples retained after filtering.

Dataset	Embedding Model	Poison-(100%) F1↑	Poison-(80%) F1↑ / CRR↑	Poison-(60%) F1↑ / CRR↑	Poison-(40%) F1↑ / CRR↑	Poison-(20%) F1↑ / CRR↑	Poison-(0%) CRR↑
NQ	SimCSE	97.5	92.6/92.0	94.3/91.5	84.9/89.0	3.1/86.2	85.0
	$\text{SimCSE}_{\text{w/o ROUGE}}$	91.3	83.9/93.0	72.0/69.0	64.4/68.3	35.8/54.8	52.6
	Bert	97.2	84.7/84.0	87.4/89.0	77.8/82.0	5.6/78.5	74.2
	$\text{Bert}_{\text{w/o ROUGE}}$	52.0	73.2/80.0	63.4/61.0	51.7/58.0	35.5/55.8	52.0
	BGE	98.1	90.8/92.0	96.9/93.0	89.5/91.0	3.0/86.3	87.6
	$\text{BGE}_{\text{w/o ROUGE}}$	93.8	85.0/93.0	86.7/83.0	79.9/80.7	27.5/51.5	51.4
MS-MARCO	SimCSE	95.6	84.7/88.0	84.0/80.0	71.7/73.0	4.6/72.0	70.6
	$\text{SimCSE}_{\text{w/o ROUGE}}$	89.4	77.3/84.0	69.6/60.5	58.1/61.7	17.0/47.5	52.4
	Bert	95.2	83.0/85.0	77.8/73.0	66.8/71.7	5.8/70.0	70.4
	$\text{Bert}_{\text{w/o ROUGE}}$	87.4	75.4/74.0	67.3/58.5	48.9/53.7	24.6/48.0	51.8
	BGE	94.2	87.2/88.0	84.1/73.0	73.4/69.3	5.0/66.0	66.8
	$\text{BGE}_{\text{w/o ROUGE}}$	91.4	81.5/78.0	73.2/59.0	64.9/66.3	17.9/46.5	47.8
HotpotQA	SimCSE	99.2	95.6/91.0	95.2/84.0	90.0/80.6	6.5/80.0	81.8
	$\text{SimCSE}_{\text{w/o ROUGE}}$	94.9	85.1/94.0	71.0/77.0	72.5/73.3	21.3/47.5	49.0
	Bert	99.2	89.7/88.0	85.5/75.5	83.7/79.7	2.4/78.0	76.2
	$\text{Bert}_{\text{w/o ROUGE}}$	88.5	79.4/88.0	64.1/61.0	48.1/55.3	25.8/49.5	49.0
	BGE	99.6	91.9/90.0	95.6/84.0	90.2/82.3	9.7/80.5	81.0
	$\text{BGE}_{\text{w/o ROUGE}}$	94.7	87.4/91.0	82.5/81.0	74.7/76.3	16.5/44.3	49.6

Conflict Resolution – Stage 2

Assuming that most malicious documents have been filtered in Stage 1, we can further enhance our framework by focusing on the LLM itself. Since LLMs are pre-trained on massive internet corpora, they possess inherent common sense (parametric knowledge). Inspired by [1-3], we leverage this internal knowledge and perform knowledge consolidation to derive a concise conclusion. In the final stage, we implement a self-assessment mechanism where the LLM determines whether to rely on internal knowledge or external retrieval.



[1] Sun, Zhiqing, et al. "Recitation-augmented language models." *arXiv preprint arXiv:2210.01296* (2022).
[2] Yu, Wenhao, et al. "Generate rather than retrieve: Large language models are strong context generators." *arXiv preprint arXiv:2209.10063* (2022).
[3] Wang, Fei, et al. "Astute rag: Overcoming imperfect retrieval augmentation and knowledge conflicts for large language models." ACL 2025.

Result

Models	Defense	HotpotQA (Yang et al. 2018)				NQ (Kwiatkowski et al. 2019)				MS-MARCO (Bajaj et al. 2018)			
		PIA ACC ↑ / ASR ↓	PoisonedRAG ACC ↑ / ASR ↓	AD ACC ↑ / ASR ↓	Clean ACC ↑	PIA ACC ↑ / ASR ↓	PoisonedRAG ACC ↑ / ASR ↓	AD ACC ↑ / ASR ↓	Clean ACC ↑	PIA ACC ↑ / ASR ↓	PoisonedRAG ACC ↑ / ASR ↓	AD ACC ↑ / ASR ↓	Clean ACC ↑
$\text{Mistral}_{\text{Nemo-12B}}$	No RAG	-	-	-	57.0	-	-	-	70.0	-	-	-	70.0
	Vanilla RAG	43.0/49.0	28.0/68.0	24.0/72.0	**78.0**	45.0/50.0	42.0/51.0	41.0/49, 0	69.0	47.0/49.0	52.0/43.0	53.0/39.0	82.0
	$\text{RobustRAG}_{\text{Keyword}}$	55.0/25.0	51.0/27.0	50.0/32.0	54.0	55.0/**4.0**	54.0/**9.0**	54.0/8.0	57.0	75.0/**6.0**	72.0/**9.0**	72.0/9.0	72.0
	$\text{InstructRAG}_{\text{ICL}}$	31.0/64.0	36.0/59.0	37.0/59.0	73.0	53.0/41.0	48.0/40.0	52.0/42.0	65.0	57.0/37.0	57.0/36.0	63.0/28.0	83.0
	ASTUTE RAG	59.0/28.0	60.0/24.0	62.0/24.0	76.0	62.0/19.0	**69.0**/10.0	66.0/**6.0**	**72.0**	72.0/24.0	77.0/16.0	84.0/**5.0**	**84.0**
	$\text{TrustRAG}_{\text{stage 1}}$	37.0/51.0	38.0/54.0	22.0/72.0	74.0	45.0/43.0	46.0/40.0	42.0/47.0	66.0	42.0/54.0	50.0/44.0	48.0/47.0	79.0
	$\text{TrustRAG}_{\text{stage 1\&2}}$	**77.0/9.0**	**74.0/13.0**	**77.0/12.0**	**78.0**	**66.0**/8.0	67.0/11.0	**67.0/6.0**	69.0	**81.0**/9.0	**84.0**/12.0	**85.0**/10.0	82.0
$\text{Llama}_{\text{3.1-8B}}$	No RAG	-	-	-	38.0	-	-	-	70.0	-	-	-	75.0
	Vanilla RAG	3.0/95.0	27.0/81.0	37.0/59.0	71.0	4.0/93.0	26.0/73.0	41.0/56.0	71.0	2.0/98.0	28.0/70.0	52.0/44.0	79.0
	$\text{RobustRAG}_{\text{Keyword}}$	55.0/4.0	40.0/50.0	49.0/49.0	54.0	44.0/11.0	51.0/27.0	61.0/24.0	61.0	69.0/15.0	67.0/19.0	71.0/15.0	72.0
	$\text{InstructRAG}_{\text{ICL}}$	64.0/27.0	54.0/41.0	46.0/51.0	**83.0**	55.0/19.0	58.0/37.0	61.0/34.0	68.0	57.0/19.0	63.0/33.0	70.0/26.0	**89.0**
	ASTUTE RAG	51.0/28.0	65.0/**16.0**	67.0/20.0	65.0	70.0/14.0	77.0/11.0	81.0/**4.0**	75.0	71.0/25.0	54.0/41.0	85.0/8.0	83.0
	$\text{TrustRAG}_{\text{stage 1}}$	28.0/61.0	43.0/47.0	36.0/59.0	70.0	40.0/52.0	43.0/50.0	45.0/52.0	65.0	31.0/67.0	45.0/47.0	43.0/52.0	81.0
	$\text{TrustRAG}_{\text{stage 1\&2}}$	**73.0/3.0**	**66.0**/18.0	**70.0/18.0**	74.0	**83.0/2.0**	**82.0/9.0**	**82.0/4.0**	**82.0**	**86.0/7.0**	**83.0/11.0**	**86.0/7.0**	85.0
GPT_{4o}	No RAG	-	-	-	64.0	-	-	-	76.0	-	-	-	80.0
	Vanilla RAG	60.0/37.0	52.0/48.0	56.0/40.0	82.0	52.0/41.0	56.0/39.0	66.0/23.0	76.0	67.0/28.0	67.0/24.0	72.0/14.0	81.0
	$\text{RobustRAG}_{\text{Keyword}}$	60.0/8.0	51.0/27.0	41.0/40.0	54.0	40.0/38.0	39.0/28.0	37.0/33.0	45.0	48.0/29.0	50.0/22.0	50.0/19.0	56.0
	$\text{InstructRAG}_{\text{ICL}}$	58.0/41.0	33.0/63.0	55.0/40.0	**86.0**	63.0/34.0	53.0/39.0	67.0/24.0	79.0	69.0/28.0	59.0/35.0	71.0/18.0	81.0
	ASTUTE RAG	74.0/16.0	78.0/22.0	80.0/**4.0**	80.0	81.0/4.0	82.0/6.0	77.0/**2.0**	81.0	86.0/11.0	77.0/13.0	**86.0/2.0**	85.0
	$\text{TrustRAG}_{\text{stage 1}}$	56.0/37.0	54.0/46.0	52.0/44.0	76.0	49.0/41.0	57.0/35.0	60.0/28.0	76.0	63.0/35.0	62.0/24.0	72.0/19.0	77.0
	$\text{TrustRAG}_{\text{stage 1\&2}}$	**83.0/3.0**	**84.0/6.0**	**85.0**/6.0	84.0	**83.0/1.0**	**83.0/4.0**	**85.0/2.0**	**84.0**	**91.0/1.0**	**86.0/8.0**	**86.0**/7.0	**89.0**

Table 1: The result table presents the performance of various defense frameworks applied to different LLMs integrated with RAG systems, primarily against single injection attacks. The best performance is in bold.

Result

Table 10: Ablaiton Studies

Dataset	Defense	Poison-(100%) ACC ↑ / ASR ↓	Poison-(80%) ACC ↑ / ASR ↓	Poison-(60%) ACC ↑ / ASR ↓	Poison-(40%) ACC ↑ / ASR ↓	Poison-(20%) ACC ↑ / ASR ↓	Poison-(0%) ACC ↑
NQ	Vanilla RAG	2.0/98.0	2.0/98.0	3.0/97.0	4.0/93.0	26.0/73.0	71.0
	$\text{TrustRAG}_{\text{w/o K-Means}}$	55.0/39.0	55.0/41.0	58.0/36.0	63.0/28.0	69.0/18.0	81.0
	$\text{TrustRAG}_{\text{w/o Conflict Resolution}}$	67.0/6.0	51.0/19.0	56.0/3.0	62.0/2.0	43.0/50.0	65.0
	$\text{TrustRAG}_{\text{w/o Internal Knowledge}}$	75.0/3.0	67.0/11.0	65.0/4.0	67.0/3.0	50.0/34.0	64.0
	$\text{TrustRAG}_{\text{w/o Self Assessment}}$	78.0/**2.0**	75.0/7.0	80.0/2.0	80.0/2.0	69.0/21.0	78.0
	TrustRAG	**83.0**/**2.0**	**85.0**/**1.0**	**84.0**/**1.0**	**83.0**/**1.0**	**82.0**/**9.0**	**82.0**
HotpotQA	Vanilla RAG	1.0/99.0	2.0/97.0	6.0/94.0	5.0/94.0	27.0/81.0	71.0
	$\text{TrustRAG}_{\text{w/o K-Means}}$	41.0/56.0	42.0/57.0	49.0/48.0	53.0/44.0	61.0/34.0	73.0
	$\text{TrustRAG}_{\text{w/o Conflict Resolution}}$	54.0/6.0	61.0/12.0	**72.0**/**3.0**	66.0/**2.0**	43.0/47.0	**81.0**
	$\text{TrustRAG}_{\text{w/o Internal Knowledge}}$	**69.0**/6.0	61.0/10.0	67.0/6.0	**72.0**/8.0	51.0/32.0	67.0
	$\text{TrustRAG}_{\text{w/o Self Assessment}}$	64.0/5.0	59.0/8.0	65.0/6.0	67.0/5.0	55.0/32.0	65.0
	TrustRAG	67.0/**4.0**	**71.0**/**4.0**	70.0/7.0	69.0/5.0	**66.0**/**18.0**	74.0
MS-MARCO	Vanilla RAG	3.0/97.0	3.0/96.0	5.0/94.0	7.0/93.0	28.0/70.0	79.0
	$\text{TrustRAG}_{\text{w/o K-Means}}$	51.0/47.0	53.0/44.0	53.0/42.0	64.0/32.0	83.0/11.0	**86.0**
	$\text{TrustRAG}_{\text{w/o Conflict Resolution}}$	77.0/7.0	64.0/18.0	72.0/7.0	78.0/6.0	45.0/47.0	70.0
	$\text{TrustRAG}_{\text{w/o Internal Knowledge}}$	80.0/6.0	75.0/13.0	73.0/10.0	69.0/12.0	54.0/35.0	75.0
	$\text{TrustRAG}_{\text{w/o Self Assessment}}$	86.0/**4.0**	78.0/12.0	80.0/8.0	**86.0**/**5.0**	74.0/19.0	**86.0**
	TrustRAG	**87.0**/5.0	**84.0**/**8.0**	**85.0**/**7.0**	85.0/7.0	**83.0**/**11.0**	85.0

Future Work?

(1) Designing a Mechanism for Determining When to Use Internal Knowledge

(2) Can TrustRAG defend the malicious tools in Tool-augmented LLM?

(3) Can it be applied to fact-checking scenario?

Thank you! Q&A